

 Web API

Overview

Getting started

Concepts >

► Concepts

Tutorials >

▼ Tutorials

< Tutorials

Authorization code

Authorization code PKCE

Client credentials

Implicit grant [Deprecated]

Refreshing tokens

Migration: Implicit grant to
Authorization code

Migration: Insecure redirect
URI

How-Tos >

► How-Tos

REFERENCE

Albums >

► Albums

Artists >

Refreshing tokens

A refresh token is a security credential that allows client applications to obtain new access tokens without requiring users to reauthorize the application.

[Access tokens](#) are intentionally configured to have a limited lifespan (1 hour), at the end of which, new tokens can be obtained by providing the original refresh token acquired during the authorization token request response:

```
1 {
2   "access_token": "NgCXRK...MzYjw",
3   "token_type": "Bearer",
4   "scope": "user-read-private user-read-ema
5   "expires_in": 3600,
6   "refresh_token": "NgAagA...Um_SHo"
7 }
```

Request

To refresh an access token, we must send a POST request with the following parameters:

Body Parameter	Relevance	Value
grant_type	Required	Set it to refresh_token.
refresh_token	Required	The refresh token returned from the authorization token request.

Body Parameter	Relevance	Value
client_id	Only required for the PKCE extension	The client ID for your app, available from the developer dashboard.

And the following headers:

Header Parameter	Relevance	Value
Content-Type	<i>Required</i>	Always set to application/x-www-urlencoded.
Authorization	Only required for the Authorization Code	Base 64 encoded string contains the client ID and secret key. The field must be in the format: Authorization: Basic <base64 encoded client_id:client_secret>

Example

The following code snippets represent two examples:

- A client side (browser) JavaScript function to refresh tokens issued following the [Authorization Code with PKCE extension flow](#).
- A server side (nodeJS with express) Javascript method to refresh tokens issued under the [Authorization Code flow](#).

```
1  const getRefreshToken = async () => {
2
3    // refresh token that has been previously
4    const refreshToken = localStorage.getItem
5    const url = "https://accounts.spotify.com
6
7    const payload = {
8      method: 'POST',
9      headers: {
10        'Content-Type': 'application/x-www-f
11      },
12      body: new URLSearchParams({
13        grant_type: 'refresh_token',
14        refresh_token: refreshToken,
15        client_id: clientId
16      }),
17    }
18    const body = await fetch(url, payload);
19    const response = await body.json();
20
21    localStorage.setItem('access_token', res
22    if (response.refresh_token) {
23      localStorage.setItem('refresh_token',
24    }
25  }
```

Response

If everything goes well, you'll receive a 200 OK response which is very similar to the response when issuing an access token:

```
1  {
2    access_token: 'BQBLuPRYBQ...BP8stIv5xr-Iwa
3    token_type: 'Bearer',
4    expires_in: 3600,
5    refresh_token: 'AQAQfyEFmJJuCvAFh...cG_m-2
6    scope: 'user-read-email user-read-private'
7  }
```

The refresh token contained in the response, can be used to request new tokens. Depending on the grant used to get the initial refresh token, a refresh token

might not be included in each response. When a refresh token is not returned, continue using the existing token.

DOCUMENTATION

[Web API](#)

[Web Playback SDK](#)

[Ads API](#)

[iOS](#)

[Android](#)

[Embeds](#)

[Commercial Hardware](#)

GUIDELINES

[Design](#)

[Accessibility](#)

COMMUNITY

[News](#)

[Forum](#)

LEGAL

[Developer Terms](#)

[Developer Policy](#)

[Compliance Tips](#)

[Third Party Licenses](#)